FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- $Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)$
- $Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)$

Input Format

1. The first line of input contains an integer representing the number of processes

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int n, i, completed = 0, currentTime = 0;
    float totalTAT = 0, totalWT = 0;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n], visited[n];

    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        visited[i] = 0;
    }

    while(completed < n) {
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

of processes.

2. The next lines contain two integers for each process, where the first integer denotes the Arrival Time (AT) of the process and the second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

```
4
0 6
0 8
0 7
0 3
```

Sample Output 1

Select Language : C (GCC 9.2.0)

```
19        int idx = -1;
20        int minBT = 1000000;
21
22        for(i = 0; i < n; i++) {
23            if(at[i] <= currentTime && visited[i] == 0) {
24                if(bt[i] < minBT) {
25                    minBT = bt[i];
26                    idx = i;
27                }
28            }
29        }
30
31        if(idx != -1) {
32            currentTime += bt[idx];
33            ct[idx] = currentTime;
34            tat[idx] = ct[idx] - at[idx];
35            wt[idx] = tat[idx] - bt[idx];
36
37            totalTAT += tat[idx];
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00

Average Waiting Time = 7.00

Example 2

Sample Input 2

3

0 3

5 2

5 1

Select Language : C (GCC 9.2.0)

```
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];

            totalTAT += tat[idx];
            totalWT += wt[idx];

            visited[idx] = 1;
            completed++;
        } else {
            currentTime++;  // If no process has arrived, increment time
        }
    }

    printf("\nAverage Turnaround Time = %.2f\n", totalTAT / n);
    printf("Average Waiting Time = %.2f\n", totalWT / n);

    return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1